

Knowing When to Interrupt: Threshold, a Proactive Agentic Tutor for CS1 Debugging

SAMUEL ABDULLAHI ABEEB, Colorado State University, USA

Large language models (LLMs) can solve most introductory programming (CS1) exercises, so a growing set of tutors restrict what the model says, offering hints instead of solution code. These tutors still wait for the student to ask for help. Yet novices often seek help ineffectively, and deciding when a struggling learner should receive help is a long-standing problem known as the assistance dilemma. We present Threshold, an agentic tutor for novice Python programmers that monitors execution events, detects when a student is repeatedly failing on the same runtime error, and opens a Socratic dialogue without being asked. Threshold reads the student's code to decide when and where to intervene, but its responses are limited to questions and conceptual hints. We describe a within-subjects study with 16 participants that isolates the effect of initiative: participants debugged two matched tasks with the same Socratic agent, which either initiated help on its own (proactive condition) or waited for a help request (on-request control condition). The primary outcome is performance on an unassisted transfer task; secondary outcomes are time-to-fix and perceived workload.

CCS Concepts: • **Human-centered computing** → **Interactive systems and tools**; • **Social and professional topics** → **Computing education**.

Additional Key Words and Phrases: Agentic AI, mixed-initiative interaction, proactive tutoring, assistance dilemma, productive failure, CS1, novice programmers, debugging

1 Introduction

Code-generating models can produce correct solutions to many typical introductory programming problems, which has raised concerns about student over-reliance [7]. A scoping review of 38 empirical studies of LLM use in CS1 found that balancing AI support against the development of foundational reasoning, the assistance dilemma, is a central tension for educators [3]. One response has been to build tutors with guardrails that withhold solution code and offer guidance instead [4, 13, 15]. Our research group followed this path with Drona, a retrieval-augmented coding companion that gives conceptual hints and has no access to the student's code [8].

These tutors share one design assumption: help begins when the student asks for it. Evidence suggests this assumption does not hold well for novices. When students used an LLM-powered assistant over a semester, most requests sought immediate help with assignments, and students often gave the tool minimal information about their problem [19]. In a logic tutor, a policy that offered help proactively reduced help avoidance compared with a control condition [16]. At the same time, not all struggle should be interrupted. Students who struggle with a problem before instruction can learn more deeply than students who are guided from the start [12].

This paper presents Threshold, an agentic tutor that decides for itself when to help. Threshold watches a novice's run history, detects when the same runtime error keeps recurring without progress, and responds with a Socratic question about the specific error and code region. It never writes code for the student. Threshold is a new system built from scratch; it builds on the design of Drona but shares no code or course materials with it. Our research question is whether system-initiated help improves novices' ability to debug on their own, compared with the same help offered on request. Section 2 situates Threshold in prior work, Section 3 describes the system, and Section 4 presents the study methodology.

Author's Contact Information: Samuel Abdullahi AbeeB, Department of Computer Science, Colorado State University, Fort Collins, Colorado, USA, abeeb.abdullahi@colostate.edu.

Manuscript submitted to ACM

2 Related Work

2.1 Generative AI in Introductory Programming

Denny et al. summarize evidence that code generation models solve typical introductory problems well, and discuss both risks and opportunities for CS1 [7]. On the risk side, they warn that relying too heavily on code generation tools may hinder the development of metacognitive skills. On the design side, they note that novices who over-use code generators encounter fewer errors but appear ill-equipped to handle the errors they do see. They conclude that tools must be designed to help users through the error feedback loop, and they call for guardrails so that generated output supports learning without immediately revealing solutions [7]. Aruleba et al. reach a compatible conclusion from the empirical literature. Their scoping review screened 1,383 records from four databases down to 38 empirical studies of LLMs in CS1, and identified four themes: student strategies, educator roles, ethics and equity, and pedagogical tensions centered on the assistance dilemma [3]. Both works frame the design question this project addresses: not whether novices will use AI while programming, but how AI help should be structured so that students still learn to debug.

2.2 Guardrailed LLM Tutors

Several systems restrict what the model will say. CodeHelp uses a pipeline of prompts to give on-demand help without revealing solutions. It was deployed for 12 weeks in a first-year computer and data science course with 52 students; students especially valued its availability and its help resolving errors, and the instructor found that it complemented rather than replaced human support [15]. A follow-up analysis of more than 2,500 queries from the same deployment found that most requests asked for immediate help with assignments rather than conceptual understanding, that students often supplied minimal information, and that more successful students used the tool more [19]. CourseAssist combines retrieval-augmented generation, intent classification, and question decomposition to align answers with course materials. Evaluated against GPT-4 on 50 question-answer pairs, it outperformed the baseline on usefulness, accuracy, and pedagogical appropriateness, and interviews with 20 of its more than 500 users reported faster feedback on assignments [9]. Iris, integrated into the Artemis learning platform, gives calibrated assistance through hints and counter-questions rather than complete solutions; a student survey found that students valued its immediate support and saw it as a complement to human tutors [4]. CodeAid, deployed to about 700 students for a semester, answers conceptual questions, produces explained pseudo-code, and annotates incorrect code without revealing solution code; educators valued its approach but raised concerns about occasional incorrect feedback and students defaulting to ChatGPT [13].

These evaluations differ in method, from classroom deployments [13, 15] to answer-quality benchmarks and surveys [4, 9], and most measure usage or perception rather than learning. More importantly for this project, all four systems are reactive: the student decides when to ask. Sheese et al.’s finding that students often send low-information requests [19] suggests that leaving initiative entirely to the novice limits what a guardrailed tutor can do.

2.3 The Drona Line of Work

Threshold builds on prior work in our group. Kurivella et al. tested ChatGPT-4 on all 11 labs of a CS1 course, with two researchers acting as proxy students. They then redesigned the labs with flowcharts, real-world scenarios, and diagrams, and found that flowcharts were the most effective at hindering AI-generated solutions, while ChatGPT-4 could still solve the scenario and diagram versions [14]. Because no students were tested, the authors note that prompt counts alone cannot establish that the redesigned labs increase critical thinking. Deoraj et al. then built Drona, which

uses retrieval over lab instructions to give hints and deliberately has no access to student code. Their work-in-progress study enrolled 20 students; preliminary findings from six participants suggest that guided AI can reduce cognitive load and increase persistence, especially for less experienced students, and that usage varied by experience level in ways that indicate a need for more adaptive feedback [8]. Most recently, three CS1 instructors evaluated Drona positively, noting that some students may become frustrated when the tool declines to give answers, and identified adaptive tutoring as future work [1]. Threshold pursues that direction. It also relaxes one of Drona’s constraints: to decide when to intervene, Threshold reads the student’s code and error history, while still limiting its output to questions and hints.

2.4 Productive Failure and the Assistance Dilemma

Kapur’s work on productive failure shows that struggle can be valuable. In a study with ninth-grade mathematics students learning variance, students who first tried to generate solutions without guidance, and then received instruction, significantly outperformed students taught directly on conceptual understanding and transfer, without losing procedural fluency [12]. Deciding when and whether to provide such support is a well-known challenge called the assistance dilemma [16]. Maniktala et al. addressed it in a logic-proof tutor with a data-driven HelpNeed predictor that classifies problem-solving steps as productive or unproductive. In a controlled study, an adaptive policy that gave proactive hints based on this predictor produced lower help avoidance and better posttest performance than the control [16]. In programming, Poitras et al. studied the assistance dilemma with LLM code generators in an introductory course and found that learners who were less proficient at reading and writing code were more likely to seek AI assistance [17]. These studies establish that the timing of help matters and can be estimated from student behavior. Maniktala et al.’s predictor, however, relies on prior student data in a well-structured logic domain, and its hints come from that data. Whether a similar approach works when the help itself is generated by an LLM, for novice programmers fixing runtime errors, remains untested.

2.5 Detecting Struggle and Timing Feedback

Signals of struggle in programming have long been derived from logged activity. Jadud studied novices’ edit–compile cycles and proposed the error quotient, which scores how often consecutive compilations end in the same error; he also noted that it could be computed live and suggested automatic mechanisms to help students break out of repetitive error cycles [11]. Jadud computed the error quotient from logs of Java compilation errors; Threshold applies the underlying idea, repeated failure on the same error, as an online trigger for Python runtime errors. Evidence on timing also supports acting while students are working. In a randomized controlled trial with more than 8,000 students in an online CS1 course, students who received style feedback in real time were five times more likely to view and engage with it than students who received it after a delay. Students who viewed the feedback were also more likely to make significant style-related edits, and over 79% of those edits directly incorporated the feedback [21]. That feedback addressed code style rather than stuck students, so it shows the value of timely help but not how to decide when a struggling student needs it.

2.6 Proactive AI Assistants for Programming

Proactive LLM assistants have recently appeared. Chen et al. built a chat-based programming assistant that uses the programmer’s code and history to generate suggestions, including code implementations, and decides whether each suggestion is timely enough to show. A randomized study found significant benefits alongside nuances in how design choices affect use [5]. That assistant targets productivity rather than learning, and supplying code is exactly what CS1

Table 1. Positioning of Threshold against closely related systems.

System	Domain	Initiative	Solution code	Evaluation
CodeHelp [15, 19]	Intro programming	Student	No	12-week deployment, 52 students; query analysis
CourseAssist [9]	CS courses	Student	No	Benchmark vs. GPT-4; 20 interviews
Iris [4]	CS programming exercises	Student	No	Student survey
CodeAid [13]	Intro programming	Student	No	Semester deployment, ~700 students
Drona [8]	CS1 labs	Student	No	WIP; preliminary results from 6 of 20 students
HelpNeed policy [16]	Logic proofs	System	No (data-driven hints)	Controlled study vs. control
Chen et al. [5]	Programming (productivity)	System	Yes	Randomized study of design variants
SHARP [20]	Competitive programming	System	No	LLM-simulated students; 42-student pilot, no control
Threshold	CS1 debugging	System	No	Within-subjects, 16 students; proactive vs. on-request control, same agent

tutors are designed to avoid. SHARP is the closest system to Threshold. It pairs an adaptive hint scheduler, which infers an impasse when the time or the number of non-productive attempts since the last progress event exceeds a threshold, with a Socratic hint generator that avoids revealing code [20]. In a benchmark with an LLM acting as the student, SHARP achieved higher debugging success than baseline hinting strategies. In a pilot with 42 secondary-school students, students valued its agency-preserving hints, while about 15% reported frustration with strictly indirect hints when they were severely stuck [20]. SHARP targets competitive programming, and its human pilot had no comparison condition. The Iris team has also announced plans to add proactive features that reach out to students when potential struggle is detected [2].

2.7 Positioning and Contribution

Table 1 summarizes the closest systems. Guardrailed CS1 tutors withhold solution code but leave initiative to the student. Proactive assistants take initiative but either supply code [5] or have not been compared against a request-only version of the same tutor [20]. Maniktala et al. compared proactive help with a control, but in a logic tutor with data-driven rather than LLM-generated hints [16].

This work makes three contributions:

- (1) **A proactive, guardrailed tutor for CS1 debugging.** Threshold combines the initiative of proactive assistants with the no-solution-code restraint of CS1 tutors, triggered by repeated runtime errors in novice Python programs.
- (2) **Observation separated from generation.** Unlike Drona, which withholds student code from the model [8], Threshold reads code and error history to decide when and where to intervene, while its outputs remain limited to Socratic questions and conceptual hints.
- (3) **A controlled test of initiative.** The study holds the agent, prompts, model, and hints constant and varies only who starts the dialogue, with the on-request version of the same agent as the control. The primary outcome is an unassisted transfer task, which the trigger cannot influence directly.

3 The Threshold System

Threshold is a browser-based environment with a task description and code editor on one side and an agent panel on the other. It addresses interface issues that instructors reported for Drona by using a single run control and keeping the task description visible without scrolling [1].

Execution and logging. Student code runs in the browser using Pyodide, a WebAssembly build of Python [18]. Running Python on the client means every run, its output, and any exception (type, message, and line number) are observable to the application without a round trip to a server. Each event is written to Cloud Firestore with a pseudonymous participant ID and timestamp: editor snapshots (debounced), run events and their results, and all agent events (detector triggers, messages shown, dismissals, and help-button presses).

Struggle detector. Threshold treats a student as *thrashing* when the same exception type recurs at the same source line on three consecutive runs, or when the same unresolved exception persists for three minutes across at least two runs. These thresholds were tuned in pilot sessions and then fixed before data collection (Section 4.4). The detector applies the repeated-error idea behind Jadud’s error quotient [11] online.

Socratic agent. When the detector fires in the proactive condition, or when the student presses the help button in the on-request condition, a LangChain agent uses tool calls to retrieve the relevant code region, recent run history, and a summary of the program’s abstract syntax tree. It then generates a single Socratic question or conceptual hint about the specific failure, for example: “Your list has five elements. On the last pass through the loop, what value does `i` have, and is that a valid index?” Following CodeHelp’s approach [15], every response is checked for code, and any response containing code is regenerated. The proactive prompt appears in the agent panel immediately after the failed run that triggered it, and the student can dismiss it.

4 Methodology

This section describes the study protocol in the past tense, as it will appear in the final report.¹

4.1 Research Questions and Hypotheses

RQ1 Does system-initiated Socratic help, compared with the same help offered on request, improve novices’ ability to debug on their own?

RQ2 Does system-initiated help change how quickly novices recover from errors and how much workload they perceive?

RQ3 How do novices experience being interrupted by a tutor while debugging?

We tested three hypotheses, each comparing the proactive condition with the on-request control condition:

H1 After the proactive condition, participants fix more bugs on the unassisted transfer task, and fix them faster (RQ1).

H2 In the proactive condition, participants take less time from the first occurrence of an error to its resolution (RQ2).

H3 After the proactive condition, participants report lower frustration on the NASA-TLX [10] (RQ2).

RQ3 was exploratory and was addressed through post-session interviews.

¹Data collection begins only after IRB approval. This section reports the planned protocol; participant demographics and results will be added once the study has been conducted.

4.2 Study Design

The study used a within-subjects design with one independent variable, *initiative*, at two levels:

- **Proactive (treatment).** The agent opened the dialogue when the struggle detector fired. The help button also remained available.
- **On-request (control).** The agent responded only when the participant pressed the help button.

The on-request condition served as the control. It is not a no-AI baseline: it is the same agent, with the same prompts and guardrail, used the way existing guardrailed tutors are used, where help begins when the student asks [8, 15]. Everything else was identical across conditions: the model, model version, prompts, generation parameters, tools, and output guardrail. The two conditions therefore differed only in who initiated help, so any difference between them can be attributed to initiative rather than to the model, content, or hint quality. In the control condition the detector still ran, silently, and its output was logged only for the manipulation check. Because each participant completed both conditions, each participant served as their own control.

Each participant completed one task in each condition. We counterbalanced both the order of conditions and the pairing of tasks with conditions, giving four sequences: proactive first or control first, crossed with Task A first or Task B first. Participants were assigned to sequences in rotation, four per sequence.

4.3 Participants

Sixteen undergraduate students took part, four in each counterbalancing sequence. Sixteen exceeds the course minimum of 12 participants for a repeated-measures design and allowed each of the four sequences to be filled equally. Participants were eligible if they were enrolled in, or had completed, Colorado State University’s introductory Python programming course. Students who had served as teaching assistants for that course were excluded, because their experience would exceed that of the target population. Participants were recruited through department email lists and announcements in introductory programming courses, made with the instructors’ permission. Participation was voluntary and had no effect on course grades; any compensation followed the approved IRB protocol. A background questionnaire recorded each participant’s prior programming experience and prior use of AI coding tools.

4.4 Tasks

We wrote two original debugging tasks, A and B, for this study; neither reused Drona materials or the course’s labs. Each task was a short Python program of about 40–60 lines with a written specification and a small test suite. Each contained three seeded bugs from the same three runtime-error classes: an off-by-one loop bound that raised an `IndexError`, an incorrect dictionary access that raised a `KeyError`, and a string/integer mix-up that raised a `TypeError`. The two tasks used different cover stories but matched structure. Each task had a matching *transfer item*: a shorter program with one bug from each of the same three classes, completed with the tutor turned off. Before data collection, pilot participants who were not part of the sample completed both tasks; their sessions were used to check that the tasks were comparable in difficulty and to fix the detector thresholds.

4.5 Procedure

Sessions were conducted one-on-one in a quiet room on a laptop running the Threshold environment in a web browser (Section 3); participants’ code ran in the browser through Pyodide [18], and all events were logged to Cloud Firestore under a pseudonymous participant ID. Each session lasted about 80 minutes:

Manuscript submitted to ACM

- (1) **Consent and background** (about 5 min). Participants gave informed consent and completed the background questionnaire.
- (2) **Tutorial** (about 5 min). A practice program introduced the editor, the run button, the help button, and proactive prompts.
- (3) **Block 1** (about 30 min). Participants worked on their first task under their first condition for up to 20 minutes, completed the NASA-TLX, and then worked on the matching transfer item with the tutor off for up to 7 minutes.
- (4) **Block 2** (about 30 min). The same sequence followed with the other task and the other condition.
- (5) **Interview** (about 10 min). A semi-structured interview, audio-recorded with consent, asked about each condition and about the experience of being interrupted.

4.6 Measures

- **Primary outcomes.** (1) *Transfer performance*: the number of the three transfer bugs fixed within the time limit (0–3), and the time taken to fix each one. (2) *Final task correctness*: the proportion of tests passed at the end of each task. The transfer item was completed with the tutor turned off, so the detector could not affect it directly. Final task correctness reflects whether the assisted task was solved, not how often the participant was interrupted.
- **Secondary outcomes.** (1) *Time-to-fix*: for each seeded bug, the time from the first run that raised its error to the first run in which that error no longer occurred. (2) *Workload*: raw NASA-TLX scores on its six subscales, with the frustration subscale as the focus of H3 [10].
- **Manipulation check.** The number and duration of thrashing episodes per task, the number of proactive interventions shown, and the number of help-button presses. Thrashing episodes were expected to be shorter in the proactive condition; this measure confirms that the manipulation took effect, but is not evidence of learning.
- **Process measures.** The dismissal rate of proactive prompts, and help-button use in each condition.
- **Qualitative data.** Interview transcripts, addressing RQ3.

4.7 Analysis

The study used a mixed-methods approach [6]. For participant-level outcomes, we compared the two conditions within participants using Wilcoxon signed-rank tests, which suit small samples and do not assume normally distributed differences, and we report medians, interquartile ranges, and effect sizes (r). For bug-level outcomes (whether each bug was fixed, and time-to-fix), we fit mixed-effects models with condition as a fixed effect and a random intercept for participant. Before pooling, we checked for effects of condition order and task. Because of the sample size, we treat the study as exploratory and emphasize effect sizes over significance testing. Two coders analyzed interview transcripts thematically and resolved disagreements through discussion.

4.8 Threats to Validity

- **Carryover.** Participants who received proactive questions first may have learned to question themselves in the second block. Counterbalancing distributed this effect across conditions, and we tested for an order-by-condition interaction.
- **Task equivalence.** The tasks were matched by design and checked in pilot sessions, and the task-to-condition pairing was counterbalanced.

- **Sample size and setting.** With 16 participants in a lab setting, results are exploratory and may not reflect classroom use.
- **LLM variability.** Model outputs can vary between runs. We fixed the model version and generation parameters and logged every prompt and response.
- **Detector accuracy.** The detector may fire while a student is still making progress, or miss a student who is stuck. We examined logged episodes and interview data for both kinds of error.

4.9 Ethics

The study protocol was submitted to the Colorado State University Institutional Review Board with Dr. Marcia Moraes as principal investigator, and no data were collected before approval. All participants gave informed consent and could withdraw at any time. Data were stored under pseudonymous IDs in an access-restricted database, and code snapshots were used only for this research, never for grading.

Use of Generative AI

Generative AI (Claude, Anthropic) was used to assist with drafting and editing portions of this paper, including the Related Work section, to identify and verify candidate references, and to format the bibliography and LaTeX source. The author reviewed and revised all AI-assisted text, checked each cited work, and takes full responsibility for the content.

References

- [1] Samuel Abdullahi, Rajeshwari Deoraj, and Marcia Moraes. 2026. Perspective of CS1 Instructors about Drona Coding Companion Application. (2026). Manuscript submitted to ICETC 2026.
- [2] Applied Education Technologies Group, Technical University of Munich. 2026. Iris: An AI-Driven Virtual Tutor for Computer Science Education (project page). <https://aet.cit.tum.de/projects/edtech/iris/>. Accessed Sep. 28, 2026.
- [3] Kehinde Aruleba, Solomon Sunday Oyelere, George R. Obaido, and Ismaila Temitayo Sanusi. 2025. A Scoping Review of Student and Educator Engagement with Large Language Models in Introductory Programming Education. In *Proceedings of the 2025 Conference on UK and Ireland Computing Education Research (UKICER '25)*. Association for Computing Machinery, New York, NY, USA, 1–8.
- [4] Patrick Bassner, Eduard Frankford, and Stephan Krusche. 2024. Iris: An AI-Driven Virtual Tutor for Computer Science Education. In *Proceedings of the 2024 Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 394–400. doi:10.1145/3649217.3653543
- [5] Valerie Chen, Alan Zhu, Sebastian Zhao, Hussein Mozannar, David Sontag, and Ameet Talwalkar. 2025. Need Help? Designing Proactive AI Assistants for Programming. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 881, 18 pages.
- [6] John W. Creswell and Vicki L. Plano Clark. 2017. *Designing and Conducting Mixed Methods Research* (3rd ed.). Sage Publications, Thousand Oaks, CA, USA.
- [7] Paul Denny, James Prather, Brett A. Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N. Reeves, Eddie Antonio Santos, and Sami Sarsa. 2024. Computing Education in the Era of Generative AI. *Commun. ACM* 67, 2 (2024), 56–67. doi:10.1145/3624720
- [8] Rajeshwari Deoraj, Jaya Veera Surendra Gupta Kurivella, and Marcia Moraes. 2025. WIP: Guiding, Not Giving: Analyzing the Effectiveness of Guided Problem-Solving in CS1. In *2025 IEEE Frontiers in Education Conference (FIE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–5.
- [9] Ty Feng, Sa Liu, and Dipak Ghosal. 2024. CourseAssist: Pedagogically Appropriate AI Tutor for Computer Science Education. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference V. 2 (SIGCSE Virtual 2024)*. Association for Computing Machinery, New York, NY, USA, 310–311.
- [10] Sandra G. Hart and Lowell E. Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of Empirical and Theoretical Research. In *Human Mental Workload*, Peter A. Hancock and Najmedin Meshkati (Eds.). Advances in Psychology, Vol. 52. North-Holland, Amsterdam, 139–183. doi:10.1016/S0166-4115(08)62386-9
- [11] Matthew C. Jadud. 2006. Methods and Tools for Exploring Novice Compilation Behaviour. In *Proceedings of the Second International Workshop on Computing Education Research (ICER '06)*. Association for Computing Machinery, New York, NY, USA, 73–84.
- [12] Manu Kapur. 2012. Productive Failure in Learning the Concept of Variance. *Instructional Science* 40, 4 (2012), 651–672. doi:10.1007/s11251-012-9209-6
- [13] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Z. Henley, Paul Denny, Michelle Craig, and Tovi Grossman. 2024. CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs. In *Proceedings of the 2024 CHI*

Manuscript submitted to ACM

- Conference on Human Factors in Computing Systems (CHI '24). Association for Computing Machinery, New York, NY, USA, Article 650, 20 pages. doi:10.1145/3613904.3642773
- [14] Jaya Veera Surendra Gupta Kurivella, Rajeshwari Deoraj, and Marcia Moraes. 2025. Designing CS Assignments to Foster Cogitation in the AI Era. In *2025 IEEE Frontiers in Education Conference (FIE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–9.
- [15] Mark Liffiton, Brad E. Sheese, Jaromir Savelka, and Paul Denny. 2024. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, Article 8, 11 pages. doi:10.1145/3631802.3631830
- [16] Mehak Maniktala, Christa Cody, Amy Isvik, Nicholas Lytle, Min Chi, and Tiffany Barnes. 2020. Extending the Hint Factory for the Assistance Dilemma: A Novel, Data-driven HelpNeed Predictor for Proactive Problem-solving Help. *Journal of Educational Data Mining* 12, 4 (2020), 24–65. doi:10.5281/zenodo.4399683
- [17] Eric Poitras, Brent Crane, and Angela Siegel. 2024. Generative AI in Introductory Programming Instruction: Examining the Assistance Dilemma with LLM-Based Code Generators. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference V. 1 (SIGCSE Virtual 2024)*. Association for Computing Machinery, New York, NY, USA, 186–192. doi:10.1145/3649165.3690111
- [18] Pyodide contributors. 2026. Pyodide: Python in the Browser via WebAssembly. <https://pyodide.org>. Accessed Sep. 28, 2026.
- [19] Brad Sheese, Mark Liffiton, Jaromir Savelka, and Paul Denny. 2024. Patterns of Student Help-Seeking When Using a Large Language Model-Powered Programming Assistant. In *Proceedings of the 26th Australasian Computing Education Conference (ACE '24)*. Association for Computing Machinery, New York, NY, USA, 49–57. doi:10.1145/3636243.3636249
- [20] Yuchen Wang, Chiraag Singh Anand, and Chee Wei Tan. 2026. Guiding, Not Solving: Agentic Scaffolding for LLM Programming Tutors. In *HAI-Agency: Workshop on Orchestrating Human and AI Agency for Proactive and Reflective Learning, co-located with AIED 2026 (CEUR Workshop Proceedings, Vol. 4248)*. CEUR-WS.org, Aachen, Germany. Paper 1. <https://ceur-ws.org/Vol-4248/paper1.pdf>
- [21] Juliette Woodrow, Ali Malik, and Chris Piech. 2024. AI Teaches the Art of Elegant Coding: Timely, Fair, and Helpful Style Feedback in a Global Course. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. Association for Computing Machinery, New York, NY, USA, 1442–1448. doi:10.1145/3626252.3630773